

Atividades Teóricas — Docker e Docker Compose (Seção 10.1)

1. Diferencie imagem e container usando uma analogia técnica e um exemplo com Node.js.

Uma **imagem** é um modelo imutável, somente leitura, que contém tudo o que uma aplicação precisa para rodar: sistema base, runtime, dependências e arquivos da aplicação. Um **container** é uma instância em execução dessa imagem — um processo isolado que roda a partir dela.

Analogia técnica: a imagem é como uma *classe* em programação orientada a objetos, e o container é como uma *instância* (objeto) criada a partir dessa classe. Assim como é possível criar vários objetos a partir da mesma classe, cada um com seu próprio estado em memória, é possível criar vários containers a partir da mesma imagem, cada um rodando isoladamente, com seu próprio sistema de arquivos temporário, processos e uso de rede — mas todos compartilhando a mesma “definição” original.

Exemplo com Node.js: a imagem `node:24-alpine` contém o binário do Node.js, o gerenciador `npm` e as bibliotecas mínimas do sistema operacional (Alpine Linux). Quando executamos:

```
docker run node:24-alpine node -v
```

o Docker cria um **container** a partir dessa imagem, executa o comando `node -v` dentro dele e o encerra. Se rodarmos o mesmo comando dez vezes, teremos dez containers diferentes (cada um com seu próprio ID), mas todos originados exatamente da mesma imagem — sem nenhuma instalação repetida do Node.js.

2. Explique por que os containers não substituem boas práticas de segurança, versionamento e observabilidade.

Containers resolvem um problema específico — **empacotamento e isolamento de execução** — mas não eliminam riscos e responsabilidades que já existiam antes deles. Colocar uma aplicação em um container não a torna automaticamente segura, rastreável ou monitorável.

- **Segurança:** uma imagem com vulnerabilidades no sistema base, dependências desatualizadas ou rodando como usuário `root` continua vulnerável dentro de um container. O isolamento do Docker reduz o “raio de impacto” de um ataque, mas não corrige falhas de código, senhas fracas, portas expostas desnecessariamente ou segredos versionados em texto puro.
- **Versionamento:** containers não substituem Git nem controle de versão de código e configuração. Uma imagem `latest` sem tag fixa pode mudar de conteúdo sem aviso, quebrando builds — por isso o versionamento disciplinado (do código-fonte e das imagens, com tags específicas) continua sendo responsabilidade da equipe.

- **Observabilidade:** um container que “sobe” (status Up) não significa que a aplicação dentro dele está saudável. Sem logs estruturados, métricas e healthchecks bem definidos, é impossível saber se o serviço está realmente funcionando corretamente ou apenas “vivo” no sentido mais básico do processo.

Em resumo: Docker padroniza *como* a aplicação roda, mas continua sendo função da equipe garantir *que* ela roda de forma segura, rastreável e observável.

3. Compare volume nomeado e bind mount. Em qual caso cada um é mais adequado?

Aspecto	Volume nomeado	Bind mount
Gerenciamento	Gerenciado inteiramente pelo Docker (docker volume create)	Aponta diretamente para uma pasta do sistema de arquivos do host
Localização	Fica dentro da área interna do Docker (ex: /var/lib/docker/volumes/...)	Fica em um caminho explícito escolhido pelo usuário na máquina host
Portabilidade	Alta — não depende da estrutura de pastas do host	Baixa — depende do caminho existir exatamente daquele jeito na máquina
Caso de uso típico	Persistência de dados de bancos de dados (ex: MySQL, PostgreSQL)	Código-fonte em desenvolvimento, para hot reload
Performance no Windows/macOS	Geralmente melhor, pois é otimizado pelo Docker	Pode ser mais lento, pois passa pela camada de compartilhamento de arquivos da VM

Quando usar cada um:

- **Volume nomeado** é mais adequado para **dados que precisam persistir e sobreviver à recriação de containers**, como o diretório de dados de um banco (mysql_data:/var/lib/mysql). O volume não depende de nenhuma estrutura de pastas específica da máquina do desenvolvedor, o que o torna mais portátil entre Windows, Linux e macOS.
- **Bind mount** é mais adequado para **desenvolvimento ativo de código**, onde o objetivo é que alterações feitas no editor (fora do container) reflitam instantaneamente dentro do container — por exemplo, ./backend:/app, permitindo hot reload sem precisar reconstruir a imagem a cada mudança.

4. Explique por que localhost dentro de um container não representa o host nem outro container.

Cada container Docker possui sua **própria stack de rede isolada** (namespace de rede do Linux). Isso significa que, dentro de um container, localhost (ou 127.0.0.1) se refere **ao próprio container** — e não à máquina física que está rodando o Docker, nem a nenhum outro container.

Por isso, se dentro do container api tentarmos acessar `http://localhost:3306` esperando encontrar o container mysql, a conexão falhará (ou irá para uma porta 3306 dentro do próprio container api, que provavelmente não existe), porque o MySQL não está rodando *dentro* do container api — está em outro container, com seu próprio localhost isolado.

A forma correta de um container se comunicar com outro, quando ambos estão na mesma rede Docker (networks), é usando o **nome do serviço** como se fosse um hostname — o Docker Compose cria automaticamente uma resolução de DNS interna para isso:

```
environment:
  DB_HOST: mysql # nome do serviço no docker-compose.yml, não
"localhost"
```

Esse é o motivo pelo qual, no exemplo do Cenário 1 do material, a api se conecta ao banco usando mysql como host, e não localhost — o Compose resolve mysql para o IP interno correto daquele container dentro da rede app_net.

5. Explique a diferença entre Redis como cache e RabbitMQ como broker de mensagens.

Embora ambos frequentemente apareçam juntos em arquiteturas modernas, eles resolvem problemas diferentes:

Redis (cache):

- É um banco de dados **em memória**, extremamente rápido para leitura e escrita.
- Usado para **armazenar temporariamente** resultados que são caros de recalcular ou buscar repetidamente (ex: resultado de uma consulta pesada no banco, sessões de usuário, contadores, rate limiting).
- O modelo de acesso é tipicamente **pergunta/resposta síncrona**: a aplicação pergunta “esse dado está em cache?” e, se estiver, usa; se não, busca na fonte original e opcionalmente guarda no Redis para a próxima vez.
- Perder os dados do Redis normalmente não é catastrófico — na pior das hipóteses, a aplicação volta a buscar do banco de dados original, só fica temporariamente mais lenta.

RabbitMQ (broker de mensagens):

- É um sistema de **filas assíncronas**, baseado no protocolo AMQP.
- Usado para **desacoplar** partes do sistema no tempo: um serviço “publica” uma mensagem (ex: “pedido criado”) e outro serviço (“worker”) a consome depois, no seu próprio ritmo, sem que o primeiro precise esperar a resposta.
- Ideal para tarefas que não precisam de resposta imediata: envio de e-mail, processamento de imagem, geração de relatório, integração entre microsserviços.
- Perder mensagens de uma fila pode ser crítico (ex: perder um “pedido criado” sem processar o pagamento), por isso o RabbitMQ tem mecanismos de persistência e confirmação de entrega que o

Redis, como cache simples, normalmente não prioriza.

Resumindo com uma frase: Redis acelera o que já vai acontecer de qualquer forma (leitura de dados); RabbitMQ organiza o que ainda vai acontecer, de forma assíncrona e desacoplada (processamento de eventos/tarefas).

6. Discuta por que o latest pode ser perigoso em ambientes profissionais.

A tag latest é apenas uma **convenção** — não significa “a versão mais recente e estável”, mas sim “a última imagem publicada sem uma tag específica” (ou explicitamente marcada como latest pelo mantenedor). Usar latest em produção ou em ambientes de equipe traz vários riscos:

1. **Falta de reprodutibilidade:** dois desenvolvedores que rodam `docker pull mysql:latest` em dias diferentes podem baixar versões **diferentes** da imagem, causando o clássico problema de “funciona na minha máquina, mas não na do colega”.
2. **Quebras inesperadas:** uma atualização da imagem base pode introduzir mudanças de comportamento, remoção de variáveis de ambiente suportadas, ou até vulnerabilidades novas — sem que a equipe tenha decidido conscientemente fazer essa atualização.
3. **Dificuldade de rollback:** se algo quebrar em produção após um `docker pull`, é difícil saber exatamente qual versão estava rodando antes, porque latest não guarda esse histórico de forma clara.
4. **Auditoria e segurança comprometidas:** times de segurança não conseguem rastrear com precisão qual versão de uma dependência está rodando em cada ambiente.

A prática recomendada é **fixar versões explícitas** (ex: `mysql:8.4`, `redis:7.4-alpine`, `node:24-alpine`), como já é feito nos exemplos de Compose do próprio material — isso garante que o ambiente seja **determinístico**: a mesma imagem, com o mesmo conteúdo, é usada por todos os membros da equipe e em todos os ambientes (dev, homologação, produção), até que a atualização seja feita de forma deliberada.

7. Explique como o Compose melhora o onboarding de novos desenvolvedores.

Sem Docker Compose, um novo desenvolvedor entrando em um projeto normalmente precisaria instalar manualmente, na própria máquina, todas as dependências do sistema: versão correta do Node.js, do MySQL, do Redis, do RabbitMQ, configurar cada um deles, criar bancos e usuários, ajustar variáveis de ambiente — um processo longo, propenso a erros e diferente em cada sistema operacional (Windows, Linux, macOS).

Com o Docker Compose, todo esse conhecimento fica **declarado em um único arquivo versionado** (`docker-compose.yml`), que descreve exatamente quais serviços existem, quais versões de imagem usar, como eles se conectam entre si e quais variáveis são necessárias. O onboarding se resume, na prática, a:

```
git clone <repositorio>
cp .env.example .env
docker compose up -d --build
```

Isso traz benefícios diretos: - **Redução de tempo de setup:** de horas/dias instalando serviços manualmente para minutos rodando um comando. - **Consistência entre máquinas:** o mesmo docker-compose.yml gera o mesmo ambiente, independentemente do sistema operacional do desenvolvedor. - **Documentação viva:** o próprio arquivo de Compose já documenta, de forma técnica e sempre atualizada, quais serviços, portas e dependências o projeto tem — evitando que essa informação fique desatualizada em um documento separado. - **Menos “poluição” na máquina do desenvolvedor:** não é necessário instalar MySQL, Redis ou RabbitMQ diretamente no sistema operacional, evitando conflitos de versão com outros projetos.

8. Liste três riscos ao versionar .env com credenciais reais.

1. **Vazamento de segredos em repositórios públicos ou privados comprometidos:** se o .env real for commitado no Git, qualquer pessoa com acesso ao repositório (incluindo, em caso de vazamento futuro do repositório, atacantes) passa a ter acesso a senhas de banco, chaves de API e tokens — mesmo que o arquivo seja removido depois, ele continua no **histórico** do Git, a menos que se reescreva o histórico manualmente (o que é complexo e arriscado).
2. **Uso indevido de credenciais de produção em ambientes de teste ou por pessoas não autorizadas:** se as mesmas credenciais reais (ex: senha do banco de produção) estiverem no .env versionado, qualquer desenvolvedor da equipe — inclusive estagiários ou colaboradores temporários — passa a ter acesso a dados sensíveis de produção sem necessidade, violando o princípio de menor privilégio.
3. **Dificuldade de rotação de credenciais:** quando uma senha ou chave precisa ser trocada (por política de segurança ou após um incidente), se ela estiver espalhada em vários commits e branches do repositório, fica muito mais difícil garantir que a credencial antiga foi completamente removida de todos os lugares — aumentando a janela de exposição.

A prática recomendada, como o material reforça, é versionar apenas um **.env.example** com os nomes das variáveis necessárias (sem valores reais ou com valores fictícios), mantendo o .env real fora do controle de versão (via .gitignore) e gerenciando os segredos reais por outro meio, como um cofre de segredos (secrets manager) ou variáveis de ambiente configuradas diretamente no servidor/CI.